

Step 5 — Backend Setup (FastAPI)

1. Introduction

Backend setup is the process of preparing the server-side environment for the application. In this project, the backend is built using **Python** and **FastAPI**. The backend handles user authentication, database operations, API requests, security, and communication with the frontend.

Before starting backend development, required Python packages must be installed.

2. Install Required Backend Packages

Open the terminal inside the **backend folder** and run the following command:

```
pip install fastapi uvicorn sqlalchemy bcrypt pyjwt python-multipart
```

Also install:

```
pip install passlib[bcrypt]
```

3. Explanation of Each Package

3.1 FastAPI

FastAPI is a modern Python web framework used to build APIs quickly and efficiently. It helps developers create backend services with high performance.

Uses of FastAPI:

- Creates REST APIs
- Handles frontend and backend communication
- Manages routes and requests
- Supports automatic API documentation

Example:

A login API or signup API can be created using FastAPI.

3.2 Uvicorn

Uvicorn is an ASGI server used to run FastAPI applications.

Without Uvicorn, the FastAPI backend cannot start.

Uses of Uvicorn:

- Runs the backend server
- Reloads automatically during development
- Makes the application accessible in the browser

Run Command:

```
uvicorn app.main:app --reload
```

Explanation of Command:

- `app` → backend folder name
 - `main` → `main.py` file name
 - `app` → FastAPI object name
 - `--reload` → Automatically restarts server after code changes
-

3.3 SQLAlchemy

SQLAlchemy is a Python library used for database management.

It connects the FastAPI backend with databases and performs operations such as storing, updating, deleting, and retrieving data.

Uses of SQLAlchemy:

- Database connection
- Table creation
- Data insertion and retrieval
- Query execution

Example:

Store user details like:

- Name
- Email
- Password

inside the database.

3.4 Bcrypt

Bcrypt is used for password encryption (hashing).

Instead of saving normal passwords in the database, passwords are converted into encrypted form for security.

Why use Bcrypt?

If the database is leaked, original passwords remain protected.

Example:

Normal Password:

mypassword123

Encrypted Password:

\$2b\$12\$KJHFGSDFGHJKL...

3.5 PyJWT

PyJWT (Python JSON Web Token) is used for authentication.

It creates secure login tokens after successful login.

Uses of JWT:

- User login authentication
- Session management
- Secure API access

Workflow:

1. User logs in
 2. Backend verifies credentials
 3. JWT token is generated
 4. User stays authenticated
-

3.6 Python-Multipart

Python-Multipart is used for handling form data and file uploads.

It helps when users upload:

- CSV files
- Excel datasets
- Images
- Documents

Uses:

- File upload feature
 - Form data handling
-

3.7 Passlib with Bcrypt

Passlib is a password hashing library.

When used with Bcrypt, it makes password encryption easier and more secure.

Installation:

```
pip install passlib[bcrypt]
```

Benefits:

- Secure password storage
 - Easy password verification
 - Better authentication security
-

4. Why These Packages Are Important

These packages together help build a secure and scalable backend system.

Package	Purpose
FastAPI	Create backend APIs
Uvicorn	Run FastAPI server
SQLAlchemy	Database operations
Bcrypt	Encrypt passwords
PyJWT	Authentication token
Python-Multipart	File uploads
Passlib	Password hashing

5. Conclusion

Installing these packages is the first step in backend development. They provide functionality for server execution, database connection, authentication, password security, and file handling. After installation, the backend becomes ready for API development and database integration.